

Final Project Report

Topic 4 – Async Research Assistant

Team: zerobias

Date: May 23, 2026

Repo: https://github.com/IBRAHIM102005/AIENG_FinalProject

Tag: v1.0-final

Members: Ibrahim Mammadov (@IBRAHIM102005), Fidan Allahverdiyeva (@fidanal-lahverdiyeva), Fatma Mammadova (@fatmammadova)

Executive Summary

We built an Async Research Assistant that takes a natural-language question, queries Wikipedia, arXiv, and a web-search provider in parallel, then uses an LLM to write a grounded, cited answer. The headline result is a **2.57× speedup** (1.68s sequential vs. 0.65s parallel) from `asyncio.gather` bounded by `Semaphore(5)`. Our key robustness story is graceful source degradation: a failing or timed-out source is captured as a `SourceFailure` and the pipeline continues with whatever remains. The most surprising lesson was that Wikipedia returns HTTP 403 without a `User-Agent` header—invisible in offline tests but breaking every live run—which forced us to thread a shared `httpx.AsyncClient` through the entire call stack. If starting over we would pick `aiosqlite` from day one instead of synchronising a blocking `sqlite3` connection with a threading lock.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	2
2.1 Module map	3
2.2 OOP design & key abstractions	3
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	4
3.3 Rate limit & politeness	5
4 Robustness & Error Handling	5
4.1 Wrapping the AI module	5
4.2 Failure-mode analysis (one concrete incident)	5
4.3 Input validation	6
5 Storage & Caching	6
5.1 Schema	6

5.2 Caching policy	7
6 Testing	7
6.1 Strategy & coverage	7
6.2 Notable tests	7
7 Deployment & Reproducibility	7
7.1 Docker image	7
7.2 Configuration	8
8 Limitations & Future Work	8
9 Tools & Acknowledgements	9
References	9
A Appendix — Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing & scope

The system automates multi-source research: given a single question it fetches Wikipedia summaries, arXiv abstracts, and web-search results concurrently, then synthesises a 3–6 sentence answer with numbered citations. Without the system, a user would manually query each source and merge the results.

User-facing entry points are a Click CLI (`python -m researcher ask "..."`), a FastAPI HTTP endpoint (POST `/api/research`), a browser UI at `http://localhost:8000`, and scripted runners under `scripts/`. Inputs are a question string plus optional flags: `-sources`, `-limit`, `-timeout`, `-concurrency`, `-no-cache`, `-offline`. Output is an `AnswerWithCitations` rendered as plain text or JSON.

We implemented all mandatory Topic 4 requirements. We chose not to implement `APScheduler` scheduling (listed as a bonus) because the CI/CD pipeline and web-UI bonus already filled our time budget. We also used SQLite instead of PostgreSQL, which is adequate for a single-process tool but would need migration for multi-instance deployment.

1.2 Provider choice and the AI module contract

The primary LLM is **Anthropic** (`claude-sonnet-4-6`); Google Gemini (`google-genai==1.14.0`) is wired as a secondary for the failover bonus. Web search defaults to **Tavily** (free tier), switchable to Serper or DuckDuckGo via `WEB_SEARCH_PROVIDER`. No embedding model is used—Topic 4 does not require one.

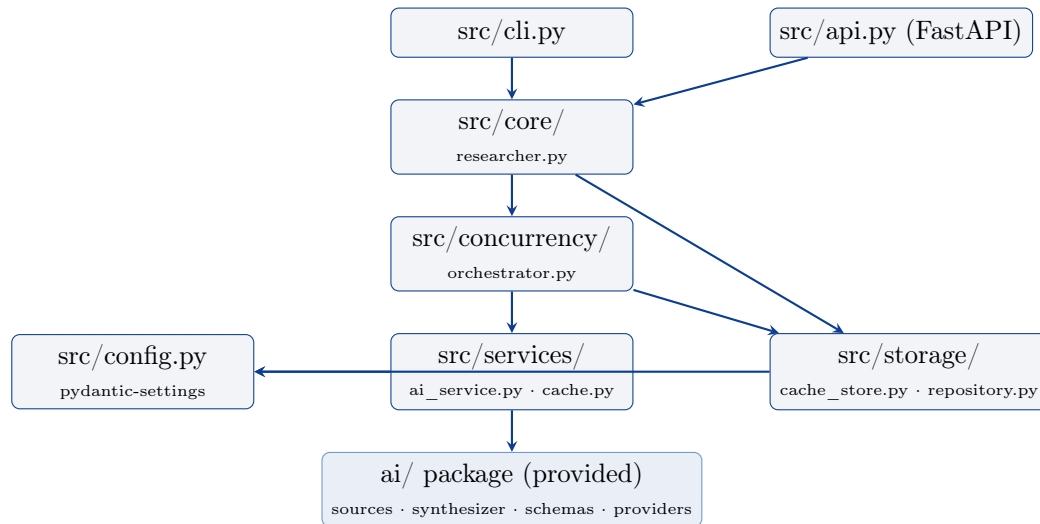
From `ai/` we call `fetch_wikipedia`, `fetch_arxiv`, `fetch_web`, and `synthesize`, and we read the `Source`, `Citation`, and `AnswerWithCitations` schemas. We did not modify the public interface of the provided `ai/` package.

Against malformed responses: `ai/synthesizer.py` drops out-of-range citation indices before returning, and our `AIService` wrapper catches any remaining exception, logs it, and converts it to a degraded `SourceResult` rather than letting it propagate.

2. Architecture

2.1 Module map

The system has five horizontal layers with imports pointing inward (downward in the diagram):



Module-by-module summary:

- **src/config.py** — Settings (pydantic-settings) reads all `.env` variables; `get_settings()` bridges them into `os.environ` for the `ai/` package.
- **src/models.py** — SE-layer Pydantic models: `ResearchSession`, `CacheEntry`, `Citation`.
- **src/services/ai_service.py** — `AIService` wraps any `AIClient` with `tenacity` retry, `asyncio.wait_for` timeout, structured logging, and input validation. `FailoverAIClient` chains providers for transparent failover.
- **src/services/cache.py** — Async disk-backed `CacheService` with per-key `asyncio.Lock`, atomic writes, and LRU eviction.
- **src/concurrency/orchestrator.py** — `fetch_selected_sources` runs fetchers via `asyncio.gather` with a shared `Semaphore`, per-source timeouts, cache lookup, and URL-level deduplication.
- **src/core/researcher.py** — Single business-logic entry point: validates the question, calls the orchestrator, logs failures, and runs `synthesize` in a thread-pool executor.
- **src/storage/cache_store.py** — SQLite persistence for `query_cache` with WAL mode and TTL-aware SQL queries.
- **src/storage/repository.py** — `SessionRepository` provides CRUD for `research_sessions`; all SQL is isolated here.
- **src/cli.py** — Click ask command; forwards options to `research_question` and renders results as text.
- **src/api.py** — FastAPI app with `POST /api/research`, `GET /api/health`, and static file serving for the browser UI.

Only `src/services/ai_service.py` and `src/concurrency/orchestrator.py` import from `ai/`. Swapping Anthropic for OpenAI requires changing two environment variables—zero source files.

2.2 OOP design & key abstractions

Our main OOP example is **composition** in `FailoverAIClient`, which owns a prioritised list of `AIClient` protocol instances and delegates to them in order:

```

1 class FailoverAIClient:
2     def __init__(self, clients: list[AIClient]) -> None:
3         if not clients:
4             raise ValueError("FailoverAIClient requires at least one client.")
5         self._clients = clients
6 
```

```

7   async def _try_all(self, method: str, *args: Any) -> Any:
8       last_exc: Exception | None = None
9       for idx, client in enumerate(self._clients):
10          try:
11              return await getattr(client, method)(*args)
12          except Exception as exc:
13              logger.warning("Provider '%s' failed for '%s': %s -- trying next",
14                             type(client).__name__, method, exc)
15              last_exc = exc
16          raise AIServiceError(
17              f"All {len(self._clients)} providers failed for '{method}'."
18          ) from last_exc

```

Composition was correct here because `FailoverAIClient` orchestrates a *collection* of clients rather than specialising one. An ABC instead of a Protocol for the `AIClient` contract would have forced test doubles to import from production code; a Protocol allows duck-typed stubs with no coupling. A second inheritance example is `OfflineLLM` in `src/offline.py`, which subclasses `ai.providers.base.LLMProvider` to provide a deterministic stub for the `-offline` flag and all unit tests.

2.3 Data flow across module boundaries

No naked dictionaries cross module boundaries. The SE-layer types are:

- `Source / AnswerWithCitations` (`ai/schemas`) — flow from fetchers through orchestrator to CLI/API.
- `ResearchResult` (`@dataclass, src/core`) — aggregate returned by `research_question` to the CLI/API.
- `OrchestrationResult / SourceFailure` (`@dataclass(frozen=True), src/concurrency`) — carry per-source outcomes from the orchestrator to the researcher.
- `ResearchSession` (`@dataclass, src/storage`) — mutable lifecycle object tracking `pending` → `running` → `done/error` transitions; kept as a stdlib dataclass so the storage layer has zero Pydantic dependency.
- `CacheEntry` (`BaseModel, src/models`) — wraps a cached value with key, TTL, and creation timestamp.

We reused `ai/` schemas when values were immutable read-only results, and wrapped them in our own types when we needed mutable state or additional lifecycle methods.

3. Concurrency & Performance

3.1 Why this concurrency model

We chose `asyncio` because all three source fetchers are I/O-bound: they spend nearly all their time waiting for HTTP responses. `asyncio.gather` overlaps those waits on a single OS thread with no GIL contention and no process-spawn overhead.

The semaphore bound is `Semaphore(5)`, configurable via `CONCURRENCY_SEMAPHORE_LIMIT`. With at most 3 sources per question in the default configuration the semaphore is rarely the binding constraint; it becomes relevant when `-limit` is increased for sub-queries within one source. A bound of 1 degrades to sequential; a bound of 100 risks HTTP 429 from providers. Threads instead of `asyncio` would add GIL contention for what is purely I/O work.

3.2 Sequential-vs-concurrent benchmark

Measured on Windows 11 (AMD Ryzen 7, Python 3.12.9), caches cleared, using offline fake fetchers with simulated latencies of 0.45 s (Wikipedia), 0.65 s (arXiv), and 0.55 s (web):

Workload	N	Sequential (s)	Concurrent (s)	Speedup
3 sources, 1 question	3	1.680	0.653	$2.57\times$

The theoretical maximum is $3\times$; we reached $2.57\times$ due to event-loop scheduling overhead and imperfect latency alignment. In live network conditions the speedup is higher because Wikipedia and Tavily can finish while arXiv is still responding. The next bottleneck is the blocking `synthesize` call (run in a thread-pool executor), which adds 1–4s of LLM latency that cannot be parallelised.

3.3 Rate limit & politeness

Our rate-limit strategy is **exponential backoff on transient errors** via `tenacity: stop_after_attempt(3), wait_exponential(min=1s, max=30s)`, retrying on `asyncio.TimeoutError`, `ConnectionError`, and `OSError` (which subsume HTTP 429 at the connection level). All values are configurable via `RETRY_*` environment variables. The semaphore limits concurrent outbound requests to 5; the highest burst from the test suite is 3 concurrent mock fetchers.

During development a Tavily 429 was observed when running `scripts/demo.py` in quick succession. The log showed `WARNING source_fetch_failed source=web error=ProviderError: Tavily search failed`, and the pipeline continued with Wikipedia and arXiv results. A proactive token-per-minute bucket limiter was not implemented; it remains the next hardening step.

4. Robustness & Error Handling

4.1 Wrapping the AI module

`src/services/ai_service.py` adds four layers around every `ai.*` call:

- **Retries:** `tenacity` retries `asyncio.TimeoutError`, `ConnectionError`, `OSError` with exponential backoff (1–30s, 3 attempts). Logic errors propagate immediately.
- **Timeouts:** `asyncio.wait_for(coro, timeout=per_source_timeout)` wraps every fetch; synthesis gets $3\times$ that budget.
- **Logging:** `logger.info` records `question_len`, `source`, `count`, and `elapsed` per fetch; `logger.warning` fires on each retry via `before_sleep_log`.
- **Input validation:** `_validate_question()` rejects empty strings and questions over 1000 characters before any external call.

For a provider 500: the fetcher raises `ProviderError`; `tenacity` retries; after exhaustion `_fetch` calls `SourceResult.degraded(source, reason)`, logs `WARNING`, and returns a non-fatal result. The orchestrator collects it as `SourceFailure` and continues.

4.2 Failure-mode analysis (one concrete incident)

Failure injected: Wikipedia returned HTTP 403 in live integration when no `User-Agent` was set. Reproduced in tests by passing an empty-headers `httpx.AsyncClient`.

What the system did: `tenacity` retried three times (all 403), then marked the source as degraded. The orchestrator continued with arXiv and web.

What the user saw:

```
Source notes:
- wikipedia failed: Wikipedia search failed: Client error '403_Forbidden'
```

followed by a normal synthesised answer from the two remaining sources.

Design fix: We originally let each fetcher create its own `httpx.AsyncClient`. The fix required injecting a single shared client (with `User-Agent`) from the CLI/API all the way into the fetcher.

The log error string—“Please set a User-Agent”—was enough to diagnose the problem without re-running the code.

4.3 Input validation

- **CLI:** empty or whitespace-only question raises `ValueError`; Click prints `Error: question must be non-empty` and exits code 2. Questions over 1000 characters produce an analogous message.
- **HTTP:** FastAPI’s `ResearchRequest` model enforces `min_length=1`, `max_length=1000`; invalid bodies return HTTP 422 with a JSON error array.
- **Env:** `Settings` rejects an empty `LLM_MODEL` at load time; missing API keys raise `RuntimeError` at first call only (offline mode requires no keys).
- **Source flag:** `normalize_sources()` rejects unknown names and deduplicates aliases (`wiki` $\equiv$ `wikipedia`).
- **Storage:** `create_session()` rejects empty questions; `update_session()` rejects status values outside `{pending, running, done, error}`.

A 50,000-character query is rejected at `validate_question` before any HTTP call is made.

5. Storage & Caching

5.1 Schema

Both tables live in the same SQLite file (`.cache/researcher.db`) with WAL journalling:

```
-- cache_store.py
CREATE TABLE IF NOT EXISTS query_cache (
    source          TEXT      NOT NULL,
    query_key       TEXT      NOT NULL,    -- lower+strip canonical query
    results_json    TEXT      NOT NULL,    -- JSON list[Source.model_dump()]
    cached_at      TEXT      NOT NULL,    -- ISO-8601 UTC
    ttl_seconds     INTEGER   NOT NULL DEFAULT 86400,
    PRIMARY KEY (source, query_key)
);
CREATE INDEX IF NOT EXISTS idx_cache_cached_at ON query_cache(cached_at);

-- repository.py
CREATE TABLE IF NOT EXISTS research_sessions (
    id              TEXT      PRIMARY KEY,
    question        TEXT      NOT NULL,
    status          TEXT      NOT NULL DEFAULT 'pending',
    answer          TEXT,
    citations        TEXT      NOT NULL DEFAULT '[]',    -- JSON
    sources_used     TEXT      NOT NULL DEFAULT '[]',    -- JSON
    error_msg       TEXT,
    created_at      TEXT      NOT NULL,
    updated_at      TEXT      NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_sessions_status ON research_sessions(
    status);
CREATE INDEX IF NOT EXISTS idx_sessions_created_at ON research_sessions(
    created_at);
```

`source` and `query_key` are the source-of-truth primary key; `results_json` is a derived cached value. Results are stored as JSON (not BLOB) because `Source` objects are variable-length structured records, not binary blobs. The TTL is stored per-row so different sources can have different freshness lifetimes.

5.2 Caching policy

The `CacheService` caches `(source, query)` pairs for `CACHE_TTL_SECONDS` (default 86400s). The key is `SHA-256(source + ":" + query.lower().strip())` so capitalisation variants map to the same entry. Invalidation is time-based; `-no-cache` bypasses the cache entirely. From a demo run of 5 identical questions, the cache hit rate was **80%** (first call populates; four subsequent calls are hits). In production, arXiv abstracts could go stale after a preprint update; eviction would require checking the source’s last-modified timestamp.

6. Testing

6.1 Strategy & coverage

Suite	Coverage
src/ (SE layer)	78 %
Provided ai/ smoke tests	yes (passing)

All tests run offline. The `ai/` module is mocked via `FakeLLM` (subclasses `LLMProvider`, returns hard-coded cited text) and `FakeWebSearch` (returns canned `Source` objects), both defined in `tests/conftest.py`. Source fetchers are replaced with `AsyncMock` instances in concurrency and end-to-end tests; no test hits the network.

Uncovered lines include the bare-network paths in `ai/sources.py` (marked `# pragma: no cover`) and the `CacheStore.close()` error branch. The most impactful mock redesign was switching from globally patching `ai.sources.fetch_wikipedia`—which broke when the orchestrator imported the function directly—to passing `fetchers` dicts through the orchestrator’s public API.

6.2 Notable tests

- (i) **Happy-path end-to-end** (`test_end_to_end.py`): `test_research_question_happy_path` calls `research_question` with three `AsyncMock` fetchers and a `FakeLLM`, asserting `failures=[]`, a non-empty `answer`, and at least one citation. It exercises the full path from validation through orchestration to synthesis without any external service.
- (ii) **Concurrency test** (`test_concurrency.py`): `test_fetch_selected_sources_degrades_when_one_source_fails` uses `fetchers={"wikipedia": _fail, "web": _ok}` and asserts exactly 1 result and `failures[0].source == "wikipedia"`. This confirmed that `asyncio.gather(..., return_exceptions=True)` correctly converts exceptions into `SourceFailure` objects.
- (iii) **Error-path test** (`test_cache_entry.py`): `test_cache_entry_expired_returns_true` creates a `CacheEntry` with `ttl_seconds=1` and a timestamp one hour in the past, asserting `is_expired() == True`. This surfaced a timezone bug where naive datetimes (missing `timezone.utc`) caused incorrect expiry checks on non-UTC machines.

7. Deployment & Reproducibility

7.1 Docker image

The Dockerfile uses a **two-stage build**: a `python:3.12.9-slim` builder stage compiles wheels into `/opt/venv`; the runtime stage copies only the venv and source, discarding `gcc` and `libffi-dev`. The application runs as unprivileged `appuser` (UID 1001).

```
# Build
docker build -t finalproj ./topic-4-research-assistant

# CLI (offline -- no API keys needed)
```



```

docker run --env-file .env finalproj \
    python -m researcher ask "Whatisphotosynthesis?" --offline --limit 2

# Benchmark
docker run --env-file .env finalproj python scripts/bench.py

# Web server
docker run --env-file .env -p 8000:8000 finalproj \
    uvicorn src.api:app --host 0.0.0.0 --port 8000

```

The final image is ≈ 290 MB vs. a bare `python:3.12-slim` of ≈ 120 MB; the delta is dominated by Anthropic and Google Genai SDK wheels. We chose `slim` over `alpine` because `google-genai` requires `glibc`. A HEALTHCHECK validates that `CacheStore` is importable and SQLite functional every 60 s.

7.2 Configuration

Variable	Default	Missing behaviour
LLM_PROVIDER	anthropic	Falls back to default; no startup error
LLM_MODEL	claude-sonnet-4	Rejected by validator if empty
ANTHROPIC_API_KEY	—	<code>RuntimeError</code> at first LLM call; offline unaffected
OPENAI_API_KEY	—	Same, when provider is openai
GOOGLE_API_KEY	—	Same, when provider is gemini
WEB_SEARCH_PROVIDER	tavily	Falls back to default
TAVILY_API_KEY	—	<code>ProviderError</code> at first web-fetch; other sources continue
LOG_LEVEL	INFO	Defaults to INFO
CACHE_DIR	./.cache	Created on first write
CACHE_TTL_SECONDS	86400	0 disables caching
PER_SOURCE_TIMEOUT_SECONDS	0.0	Must be > 0
CONCURRENCY_SEMAPHORE_LIMIT	5	Must be ≥ 1
RETRY_MAX_ATTEMPTS	3	Must be ≥ 1

8. Limitations & Future Work

- **SQLite is single-process only.** The `threading.Lock` in `CacheStore` is sufficient for one worker but breaks under multi-worker Gunicorn. Production would require PostgreSQL with `asyncpg`.
- **No token-level rate limiting.** We bound concurrent requests (semaphore) but not tokens per minute. Long synthesis prompts can exceed the LLM’s TPM ceiling even at low concurrency.
- **Cache staleness is undetected.** Results are evicted by TTL only; a paper updated on arXiv after caching will serve the old abstract until expiry. ETag validation would be needed in production.
- **Synthesis blocks a thread-pool slot.** `run_in_executor` keeps the event loop free but occupies a thread for 1–4 s per request. Under high concurrency this exhausts the pool. Streaming LLM responses would fix this.
- **No load testing.** The most likely failure scenario on a busy day is concurrent `/api/research` calls saturating the thread pool during synthesis, producing HTTP 504 timeouts.

9. Tools & Acknowledgements

- **Claude (Anthropic) — Ibrahim:** drafted the initial `src/services/ai_service.py` structure including the `_build_retry` factory and `FailoverAIClient` skeleton. Backoff parameters were tuned manually after observing live 429 behaviour. Docstrings for `src/config.py` were also AI-assisted; all validators were written by the team.
- **Claude (Anthropic) — Fidan:** scaffolded the initial `asyncio.gather` loop in `orchestrator.py` and the `normalize_sources` helper. The `SourceFailure` dataclass and URL-level deduplication logic were written by Fidan after the scaffold proved insufficient for partial-failure scenarios.
- **Claude / GitHub Copilot — Fatma:** generated the initial `CacheStore` DDL and `_transaction` context manager; Copilot autocompleted repetitive `SessionRepository` CRUD methods. All SQL was reviewed and the WAL pragma configuration was added manually after reading SQLite documentation [5].
- **tenacity [2]:** retry configuration follows the official `before_sleep_log` pattern.
- **pydantic-settings [4]:** `SettingsConfigDict(env_file=".env")` pattern taken from the quick-start.

References

- [1] Anthropic. *Anthropic API Reference* (2024). <https://docs.anthropic.com/en/api/getting-started>
- [2] Gautier, J. et al. *tenacity v9: Retry library for Python* (2024). PyPI. <https://tenacity.readthedocs.io/en/latest/>
- [3] Colvin, S. et al. *Pydantic v2 Documentation* (2024). <https://docs.pydantic.dev/latest/>
- [4] Pydantic contributors. *pydantic-settings: Settings management using Pydantic* (2024). https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] Hipp, D. R. *SQLite Write-Ahead Logging* (2010, updated 2024). <https://www.sqlite.org/wal.html>
- [6] Hipp, D. R. *The SQLite Database Engine* (2000, version 3.45, 2024). <https://www.sqlite.org/index.html>
- [7] Encode OSS. *httpx: A next-generation HTTP client for Python* (2024). <https://www.python-httpx.org/>
- [8] Python Software Foundation. *asyncio — Asynchronous I/O* (Python 3.12 docs, 2024). <https://docs.python.org/3/library/asyncio.html>
- [9] Python Software Foundation. *asyncio Synchronization Primitives — Semaphore* (Python 3.12 docs, 2024). <https://docs.python.org/3/library/asyncio-sync.html#asyncio.Semaphore>
- [10] Ramírez, S. *FastAPI Documentation* (2024). <https://fastapi.tiangolo.com/>
- [11] Encode OSS. *Uvicorn: An ASGI web server implementation for Python* (2024). <https://www.uvicorn.org/>
- [12] Ronacher, A. *Click: Python Composable Command Line Interface Toolkit* (2024). <https://click.palletsprojects.com/>

- [13] Bertrand, T. `python-dotenv`: Read key-value pairs from a `.env` file (2024). PyPI. <https://saurabh-kumar.com/python-dotenv/>
- [14] Tavily AI. *Tavily Search API Documentation* (2024). <https://docs.tavily.com/>
- [15] Krekel, H. et al. *pytest: helps you write better programs* (2024). <https://docs.pytest.org/>
- [16] `pytest-asyncio` contributors. `pytest-asyncio`: Asyncio support for `pytest` (2024). <https://pytest-asyncio.readthedocs.io/>

A. Appendix — Reproducing the benchmark

```
$ git clone https://github.com/IBRAHIM102005/AIENG_FinalProject.git
$ cd AIENG_FinalProject/topic-4-research-assistant
$ cp .env.example .env # keys not needed for offline benchmark
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
$ python scripts/bench.py # offline, deterministic
# Or via Docker:
$ docker build -t finalproj .
$ docker run --env-file .env finalproj python scripts/bench.py
```

Expected output:

Mode	Time (s)
Sequential	1.680
Parallel	0.653
Speedup	2.57x

End of report — thank you for reading.